Individual Reflection for Programming Assignment 2

1. Worst Case and Triggering Conditions

Randomised Quickselect has the worst-case $\Theta(n^2)$ time when successive pivots leave almost the entire active range to search. For distinct values, repeatedly choosing the current minimum while requesting the maximum retains $n-1$, $n-2$, $n-3$,... elements, producing quadratic scanning work. This requires a statistically unlucky sequence of random choices, which sorted and reverse-sorted arrays do not immediately force. The implementation's three-way partition handles an all-equal array in one main partition. Its iterative loop avoids recursive stack growth, but it cannot prevent excessive partitioning.

2. Hardest Design Decision and Resolution

The hardest design issue was making tests comparable while keeping each author's branch independently buildable. Both suites used the same seed (14487692), but Quickselect alternated the distributions and resetted its generator for large cases, unlike Median of Medians' one generation sequence. Matching seeds did not cause matching arrays. Only randomised_quickselect_tests.cpp was revised to reproduce the same reference fixtures, distribution, and generator state. The recorded verification passed 137,246 checks. Integration used separate test executables and a shared verifier. Performance comparisons then used identical input copies in the shared runner as well.

3. What I Would Change

I would define the shared selection contract and reusable input fixtures more clearly before implementing either algorithm. Both suites could then apply the same core checks, with additional pivot seeds for Quickselect. I would also retain the current timer boundary: selection only. Generation, copying, generator construction, correctness checks, and output are to be excluded. Independent datasets and repeated sessions would also be added earlier in the process, including repeated calls with fixed varying pivot seeds. This would help distinguish between algorithmic variation and execution noise, unlike the seven changing-seed system that was used.

4. What I Now Understand

This Programming Assignment helped me understand why partitioning requires careful treatment of unclassified values. When a value greater than the pivot is swapped with the end of the unclassified region, the incoming value would not be examined. Advancing current immediately could skip the value and invalidate the whole partition. Keeping current unchanged until that value is classified preserves the

invariant. The equal region also allows an immediate return whenever k falls within its boundaries without fully sorting the array.